

1. How do you handle job failures in an ETL pipeline?

- **Retry mechanism:** Implement automatic retries for transient issues, with exponential backoff.
 - **Error handling:** Use try-catch blocks or error-handling frameworks to log detailed error messages.
 - **Alerting:** Set up monitoring tools like Airflow, Datadog, or Prometheus to notify failures.
 - **Isolation:** Ensure failed steps can be re-executed independently without restarting the entire pipeline.
-

2. What steps do you take when a data pipeline is running slower than expected?

- **Profiling:** Identify bottlenecks in I/O, computation, or network latency.
 - **Resource allocation:** Ensure adequate CPU, memory, or parallelism (e.g., Spark executors).
 - **Optimization:** Optimize query logic, reduce shuffling in Spark, or improve SQL execution plans.
 - **Caching:** Cache intermediate results to avoid redundant computations.
 - **Scaling:** Scale horizontally by increasing cluster nodes or vertically with more powerful instances.
-

3. How do you address data quality issues in a large dataset?

- **Validation rules:** Define rules to detect outliers, null values, or invalid formats.
 - **Data profiling:** Use tools like Great Expectations or Pandas Profiling for insights.
 - **Cleansing:** Automate transformations to fix or flag bad data.
 - **Versioning:** Maintain a clean version of data for rollback purposes.
 - **Feedback loop:** Work with upstream teams to address root causes.
-

4. What would you do if a scheduled job didn't trigger as expected?

- **Check scheduler logs:** Review the logs of Airflow, Cron, or other tools.
- **Dependency chain:** Ensure all upstream dependencies completed successfully.
- **System issues:** Check for server downtimes, resource unavailability, or clock mismatches.

- **Re-trigger:** If no systemic issue is found, manually re-trigger the job.
-

5. How do you troubleshoot memory-related issues in Spark jobs?

- **Memory tuning:** Adjust Spark configurations like `executor.memory` and `driver.memory`.
 - **Broadcast variables:** Reduce memory usage by broadcasting small datasets.
 - **Reduce shuffling:** Optimize joins and aggregations to minimize shuffles.
 - **Partitions:** Ensure proper partitioning to avoid skew or large partitions.
 - **Garbage collection:** Tune JVM garbage collection parameters.
-

6. What is your approach to handling schema changes in source systems?

- **Schema evolution:** Use tools like Avro or Parquet with built-in support for schema changes.
 - **Compatibility checks:** Validate the schema change impact on downstream systems.
 - **Versioning:** Maintain schema versions and update transformation logic as needed.
 - **Testing:** Test schema changes in staging environments before production rollout.
-

7. How do you manage data partitioning in large-scale data processing?

- **Partition strategy:** Use hash-based, range-based, or time-based partitioning depending on access patterns.
 - **Data size balance:** Ensure partitions are evenly distributed to prevent skew.
 - **Pruning:** Optimize partition column selection for query filtering.
 - **Repartitioning:** Use Spark's `repartition` or `coalesce` for efficient processing.
-

8. What do you do if data ingestion from a third-party API fails?

- **Retry policy:** Implement retry with exponential backoff.
 - **Fallback mechanism:** Switch to an alternate source or cached data.
 - **Rate limits:** Ensure compliance with API throttling limits.
 - **Monitoring:** Set up alerts to detect ingestion failures promptly.
-

9. How do you resolve issues with data consistency between different data stores?

- **Sync processes:** Periodically compare row counts, checksums, or hash values.
- **Transactionality:** Use distributed transactions or tools like Kafka for consistent writes.

- **Reconciliation:** Build automated processes to reconcile differences.
 - **Replication lag:** Monitor replication delays and adjust pipelines accordingly.
-

10. How do you handle out-of-memory errors in a Hadoop job?

- **Split data:** Increase the number of splits or smaller input sizes.
 - **Configuration tuning:** Adjust `mapreduce.reduce.memory.mb` and similar parameters.
 - **Use compression:** Compress intermediate data to save memory.
 - **Optimize code:** Ensure memory-intensive operations like joins or sorts are optimized.
-

11. What steps do you take when a data job exceeds its allocated time window?

- **Analyze logs:** Pinpoint slow stages or tasks.
 - **Optimize queries:** Reduce data processed or optimize joins and filters.
 - **Increase resources:** Allocate additional CPUs or memory.
 - **Prioritize tasks:** Process time-sensitive jobs first while delaying others.
-

12. How do you manage and monitor data pipeline dependencies?

- **DAG tools:** Use Airflow or similar tools to visualize and manage dependencies.
 - **Failure propagation:** Ensure dependent jobs handle upstream failures gracefully.
 - **Alerts:** Set up monitoring for dependency completion.
 - **Retry policies:** Implement retries and backoff for transient failures.
-

13. What do you do if the output of a data transformation step is incorrect?

- **Debug transformations:** Isolate and inspect the logic for correctness.
 - **Sample data:** Verify intermediate outputs against test cases.
 - **Roll back:** Revert to the last known good state.
 - **Document fixes:** Clearly document and review fixes before redeployment.
-

14. How do you address issues with data duplication in a pipeline?

- **Deduplication:** Use techniques like primary key constraints or distinct queries.
- **Idempotency:** Design jobs to be idempotent to avoid duplicates during retries.
- **Data lineage:** Trace the root cause of duplication to fix the upstream issue.

15. How do you handle and log errors in a distributed data processing job?

- **Centralized logging:** Use tools like Elasticsearch, Kibana, or Cloud Logging.
- **Granular logs:** Log task-level errors with contextual information.
- **Alerting:** Notify errors via Slack, email, or monitoring tools.
- **Error recovery:** Implement checkpoints or savepoints for fault tolerance.